



Swiss Post Voting System

Verifier Specification

Swiss Post*

Version 1.7.1

Abstract

The Swiss Post Voting System allows citizens to vote remotely in a secure and verifiable manner. Independent auditors can check that the system worked correctly. For that means, the system generates verifiable cryptographic evidence. The auditor reviews the provided evidence using a verifier software. The necessary verifications correspond to two algorithms: VerifyConfigPhase and VerifyTally. This document details the verifications that the verifier software has to implement. At the same time, it serves as a detailed specification of the Swiss Post verifier, which is an open-source software application to verify the Swiss Post Voting System. Therefore, this document serves as a manual for developing an independent verifier software and validating or extending the Swiss Post verifier.

* Copyright 2026 Swiss Post Ltd.

Disclaimer

E-Voting Community Program Material - please follow our [Code of Conduct](#) describing what you can expect from us, the Coordinated Vulnerability Disclosure Policy, and the contributing guidelines.

Revision chart

Version	Description	Author	Reviewer	Date
0.9	First published version	OE	XM, TH	2021-09-01
0.9.1		OE, HR	XM, JS, TH	2021-10-15
0.9.2		OE, HR	XM, JS, TH	2022-02-17
1.0.0	First full version	TH, OE	XM, JS, HR	2022-06-24
1.0.1	See change log for version 1.0.1	TH, OE	XM, JS, HR	2022-08-19
1.1.0	See change log for version 1.1.0	TH, OE	XM, JS, HR	2022-10-03
1.2.0	See change log for version 1.2.0	TH, OE	XM, JS, HR	2022-10-31
1.3.0	See change log for version 1.3.0	AH, OE	XM, JS, TH	2022-12-09
1.3.1	See change log for version 1.3.1	AH, OE	XM, JS, TH	2023-02-23
1.4.0	See change log for version 1.4.0	AH, OE	XM, JS	2023-04-19
1.4.1	See change log for version 1.4.1	AH, OE	XM, JS	2023-06-16
1.5.0	See change log for version 1.5.0	AH, CC, CK, OE	XM, JS	2024-02-14
1.5.1	See change log for version 1.5.1	AH, CC, CK, OE	XM, JS	2024-03-29
1.5.2	See change log for version 1.5.2	AH, CC, CK, OE	XM, JS	2024-06-17
1.6.0	See change log for version 1.6.0	AH, CC, CK, OE	XM, JS	2025-06-20
1.7.0	See change log for version 1.7.0	AH, CC, CK	OE	2026-07-03
1.7.1	See change log for version 1.7.1	OE	AH, CS, CK	2026-08-07

Contents

Symbols	4
1 Introduction	6
1.1 The Role of the Verifier	7
1.2 Conventions	9
1.3 Context and Input Variables	9
2 The Verifier in the Swiss Post Voting System	10
2.1 Structure of the Document	10
2.2 Verification Categories	10
2.3 Channel Security and Control Component Authenticity	11
2.4 Streamable Symmetric Encryption and Decryption for Dataset Security	12
2.5 Manual Checks by the Auditors	13
2.6 Election Event Context	15
2.7 Basic Data Types	15
3 Setup Verification - VerifyConfigPhase	16
3.1 Setup - Completeness	16
3.2 Setup - Authenticity	17
3.3 Setup - Consistency	20
3.4 Setup - Integrity	25
3.5 Setup - Evidence	25
4 Final Verification - VerifyTally	30
4.1 Final - Completeness	30
4.2 Final - Authenticity	31
4.3 Final - Consistency	34
4.4 Final - Integrity	37
4.5 Final - Evidence	38
References	45
List of Algorithms	46
List of Verifications	46
List of Figures	48
List of Tables	48

Symbols

$\mathbb{A}_{Base16}$	Base16 (Hex) alphabet [7]
$\mathbb{A}_{latin}$	Extended Latin alphabet, as described in the crypto primitives specification
$\mathbb{A}_{UCS}$	Alphabet of the Universal Coded Character Set (UCS) according to ISO/IEC10646
$\mathcal{T}_1^{50}$	Domain of a string token according to the XML Schema datatype [12]: strings without tabs, carriage returns, or line feeds, with collapsed whitespace (no leading/trailing spaces and no multiple internal spaces)
$\mathcal{D}_v$	Actual voting option domain, as defined in section Encoding of Voting Options of the system specification
$\mathcal{D}_\tau$	Correctness information domain, as defined in section Encoding of Voting Options of the system specification
$\mathcal{B}^*$	Set of byte arrays of arbitrary length
bb	Ballot box ID
bb	Vector of ballot box IDs of an election event
cDec	List of partially decrypted ciphertexts
CK	Confirmation Key
CCM	Mixing control components
CCR	Return Codes control components
δ	Number of write-in options + 1 for a specific verification card set
$\delta_{\max}$	Maximum number of write-in options + 1 across all verification card sets
$\delta_{\sup}$	Maximum supported number of write-in options + 1
E1	Encrypted vote
$\widehat{\mathbf{E1}}$	Exponentiated encrypted vote
E2	Encrypted partial Choice Return Codes
ee	Election event id
g	Generator of the encryption group
$\mathbb{G}_q$	Set of quadratic residues modulo p of size q . The computational proof refers to this set as $\mathbb{Q}_p$
l_{ID}	Character length of unique identifiers
L_{votes}	List of decrypted votes
$L_{\text{decodedVotes}}$	List of decoded votes
L_{writeIns}	List of decoded write-ins
m	List of plaintext votes
$\mathbb{N}^+$	Set of strictly positive integer numbers
N_{bb}	Number of ballot boxes of an election event
N_c	Number of confirmed votes in a specific ballot box
$\hat{N}_c$	Number of mixed votes including trivial encryptions

N_E	Number of eligible voters of a specific verification card set
n	Number of voting options for a given verification card set
$n_{\max}$	Maximum number of voting options across all verification card sets
$n_{\sup}$	Maximum supported number of voting options
n_{total}	Number of distinct voting options across all verification card sets
ψ	Allowed number of selections for a given verification card set
$\psi_{\max}$	Maximum number of selections across all verification card sets
$\psi_{\sup}$	Maximum supported number of selections
$\mathbb{P}$	Set of prime numbers
$\mathbf{p}$	Vector of small prime group members
$\tilde{\mathbf{p}}$	Encoded voting options $(\tilde{p}_1, \dots, \tilde{p}_n)$, $\tilde{p}_k \in (\mathbb{G}_q \cap \mathbb{P}) \setminus g$
p	Encryption group modulus
q	Encryption group cardinality s.t. $p = 2q + 1$
σ	Semantic information $(\sigma_0, \dots, \sigma_{n-1})$, $\sigma_k \in \mathbb{A}_{UCS}^*$
$\tilde{\mathbf{v}}$	Actual voting options $(v_0, \dots, v_{n-1})$, $v_k \in \mathcal{D}_v$
$\mathbf{vcs}$	Verification card set ID
$\mathbf{vcs}$	Vector of verification card set IDs of an election event
$\mathbf{w}_{\text{id}}$	Voter's encoded write-ins $(w_{\text{id},0}, \dots, w_{\text{id},\delta-2})$
$\mathbb{Z}_q$	Set of integers modulo q
$\top$	Truth value true or successful termination
$\perp$	Truth value false or unsuccessful termination

1 Introduction

Switzerland has a longstanding tradition of direct democracy, allowing Swiss citizens to vote approximately four times a year on elections and referendums. In recent years, voter turnout hovered below 40 percent [3].

The vast majority of voters in Switzerland fill out their paper ballots at home and send them back to the municipality by postal mail, usually days or weeks ahead of the actual election date. Remote online voting (referred to as e-voting in this document) would provide voters with some advantages. First, it would guarantee the timely arrival of return envelopes at the municipality (especially for Swiss citizens living abroad). Second, it would improve accessibility for people with disabilities. Third, it would eliminate the possibility of an invalid ballot when inadvertently filling out the ballot incorrectly.

In the past, multiple cantons offered e-voting to a part of their electorate. Many voters would welcome the option to vote online - provided the e-voting system protects the integrity and privacy of their vote [4].

State-of-the-art e-voting systems alleviate the practical concerns of mail-in voting and, at the same time, provide a high level of security. Above all, they must display three properties [8]:

- Individual verifiability: allow a voter to convince herself that the system correctly registered her vote
- Universal verifiability: allow an auditor to check that the election outcome corresponds to the registered votes
- Vote secrecy: do not reveal a voter's vote to anyone

Following these principles, the Federal Chancellery defined stringent requirements for e-voting systems. The Ordinance on Electronic Voting (VEleS - Verordnung über die elektronische Stimmabgabe) and its technical annex (VEleS annex) [1] describes these requirements.

Swiss democracy deserves an e-voting system with excellent security properties. Swiss Post is thankful to all security researchers for their contributions and the opportunity to improve the system's security guarantees. We look forward to actively engaging with academic experts and the hacker community to maximize public scrutiny of the Swiss Post Voting System.

1.1 The Role of the Verifier

A verifiable e-voting system requires a verifiable process and a verification software—the *verifier*—to verify the cryptographic evidence using data published on a private or public bulletin board [6]. The specification and development of the verifier should go hand in hand with the e-voting solution, and the verifier challenges and extensively tests a protocol run [5]. Therefore, the Ordinance on Electronic Voting (VEleS) [1] defines the role of the auditors and their technical aid.

[VEleS art. 2(h)]: *auditor* means a person who checks on behalf of the canton that the ballot is correctly conducted.

[VEleS art. 5 para. 3(b)]: The auditors evaluate the proof in an observable procedure; to do so, they must use *technical aids* that are independent of and isolated from the rest of the system.

For the rest of the document, we will no longer distinguish between auditors and their technical aid; we refer to *the verifier* as both the auditor and the software used by this auditor and assume that the auditor and the technical aid are trustworthy. However, the auditors must perform certain checks manually which cannot be executed by software—see section 2.5. There may be multiple auditors, and an auditor may use various technical aids. The technical annex of the Ordinance on Electronic Voting states that at least one of the auditors and one of the technical aids is trustworthy for universal verifiability.

[VEleS Annex 2.9.2.2]: The following system participants may be considered trustworthy

- [...]
- one auditor in any group, leaving open which auditor it is
- one technical aid from a trustworthy auditor, leaving open which aid it is.

In principle, everybody could become an auditor and could check an election event's proofs and data by themselves. The auditors *represent* voters in the sense of universal verifiability as elaborated in the Federal Chancellery's explanatory report [2].

[Explanatory Report, Sec 4.2.1]: The use of auditors promotes transparency. Voters should be able to assume that auditors will draw attention to possible irregularities.

[Explanatory Report, Sec 4.2.1]: With universal verifiability, manipulations in the infrastructure can be detected. Unlike individual verifiability, it does not necessarily have to be offered to voters. Instead, auditors can be employed to apply universal verifiability.

[VEleS art. 2(i)]: Infrastructure means hardware, software [...], network elements, premises, services and equipment of any nature at any operating bodies that are required for the secure operation of electronic voting.

The auditors are subject to the following requirement:

[VEleS Annex 8.14]: The auditors should be suitably informed about and trained in the processes that determine the accuracy of the result, the preservation of voting secrecy and the avoidance of premature results (for example key generation, printing the voting papers, decryption and tallying). They must be able to understand the essential aspects of the processes and their significance.

However, the exact selection of auditors is a cantonal responsibility and out of the scope of this document:

[VEleS art. 14]: Responsibility for running the ballot with electronic voting correctly.
The canton shall appoint a body at cantonal level that bears overall responsibility, and for the following tasks in particular. [...]
h. supporting and instructing the auditors.

Swiss Post releases its verifier under a permissive *open-source license*: strengthening the independence and trustworthiness of the verifiability of the system and complying with the Ordinance:

[VEleS Annex 3.18]: The software for the auditors' technical aids must be obtained from a different system developer from the one who developed the main part of the software for the other system components. The publication of the software for the technical aid under a licence that meets the criteria for open source software may justify an exception.

1.2 Conventions

The verifier specification follows the same conventions specified in the **Conventions** section of the system specification [11].

1.3 Context and Input Variables

The verifier specification uses the same concept of context and input variables as detailed in the **Context, State, and Input Variables** section of the system specification [11]. We slightly deviate from the system specification for the consistency verifications. These verifications are very simple and as a consequence all variables are treated as input, without differentiating between context and input variables.

2 The Verifier in the Swiss Post Voting System

2.1 Structure of the Document

We distinguish two runs of the verifier according to their objectives and context. The first run happens after the configuration phase of the election event and before the voters can start submitting encrypted votes. The objective is to ensure that all parameters are valid and have been generated in a way that protects the security goals of the system.

The second run is carried out after the tally phase. At this stage, the verifications should ensure that the security properties of the system have been upheld and that the election outcome reflects the combination of the ballots submitted by the voters. This implication is further discussed in [10].

The verifications required for the first run (**VerifyConfigPhase**) are presented in section 3, while those needed for the final run (**VerifyTally**) are detailed in section 4.

Each verification run executes all verifications we document in the corresponding section and the auditors must check that every verification ran successfully. Otherwise, the auditors stop the process and a detailed analysis of the failed verification takes place. We omit a detailed pseudocode of **VerifyConfigPhase** and **VerifyTally** since their only purpose is to execute all verifications of the corresponding run.

2.2 Verification Categories

Both verifier runs contain verifications for various categories, which we will identify as proposed by Haenni et al. in [6]. Table 1 shows these categories.

Category	Description
Evidence	Are the cryptographic evidence contained in the election data all valid? Do they provide the necessary evidence to infer the correctness of corresponding protocol steps?
Authenticity	Can the data elements be linked unambiguously to the party authorized to create them?
Consistency	Are related data items consistent to each other?
Integrity	Do all data elements correspond to the specification? Are they all within the specified ranges?
Completeness	Do the data elements allow a complete verification chain?

Tab. 1: Verification categories and their description.

This document provides the pseudocode algorithms for the verification of the cryptographic *evidence*. The *authenticity* checks are based on digital signatures, with each party being identified and their signing keys known before the election starts. When necessary, the verifier specification also highlights important *consistency* checks. Since we use a mathematically precise pseudocode specification, most of the *integrity* checks consist of validating the input ranges and preconditions. Furthermore, we base our verifier specification on the computational proof of complete verifiability and privacy [10], thereby making sure that our verifications are *complete* and ensure the necessary security objectives, provided the verifier has received all elements listed in this document.

2.3 Channel Security and Control Component Authenticity

A meaningful verification of election event data must include a check that the protocol’s run involved the actual honest components. Otherwise, the adversary could impersonate protocol participants — undermining verifiability and vote secrecy.

Recall that our trust model considers the setup component and one out of four control components trustworthy. The Ordinance’s explanatory report elaborates on the term *trustworthy*.

[Explanatory Report, Sec 5.2.2]: Cryptographic protocols make it possible to reduce to a minimum the number of elements that an attacker would have to control in order to manipulate votes without being detected or violate voter secrecy. Measures to prevent an attacker from taking control of an element can therefore focus on a limited number of elements. These elements are particularly worthy of protection and, ideally, can also be protected particularly effectively. Such elements found under Numbers 2.1 and 2.2 ‘System participants’ and ‘Communication channels’ are referred to as ‘trustworthy’. This may seem surprising at first glance: why is an element that is particularly worthy of protection called ‘trustworthy’? The reason lies in the fact that cryptographic protocols are not aimed at protecting those elements. The designation ‘trustworthy’ signals to authors and readers of the document in which the cryptographic protocol is specified that they do not need to worry about possible attacks in which an attacker takes control of these elements. By being trustworthy, system participants *refuse* to cooperate with an attacker. The protocol must be defined in such a way that, as long as the trustworthy system participants adhere to the protocol, the attacker will not succeed even if they bring the remaining non-trustworthy system participants under control. The use of the term is based on the literature.

All elements received by the verifier must be signed by the expected party with the **GenSignature** algorithm from the crypto primitives specification[9] or the **GenXMLSignature** algorithm from the system specification[11]. The exact data being signed as well as the additional context data used for the signature are specified in the **Channel Security** section of the system specification[11], and are reprised in this document for each of the corresponding *authenticity* checks. In order to be able to verify the signatures (using algorithm **VerifySignature** from the crypto primitives specification or the **VerifyXMLSignature** algorithm from the system specification), the verifier must first be made aware of signature keys of each of the parties. As such, each party (*i.e.* control components, tally component and setup component) designates a responsible person that transmits their certificate out-of-band to the auditor during a certificate ceremony. The certificates are then imported into the verifier’s keystore after having been verified as per section **Importing a Trusted Certificate** from the crypto primitives specification. Figure 1 highlights the verification procedure.

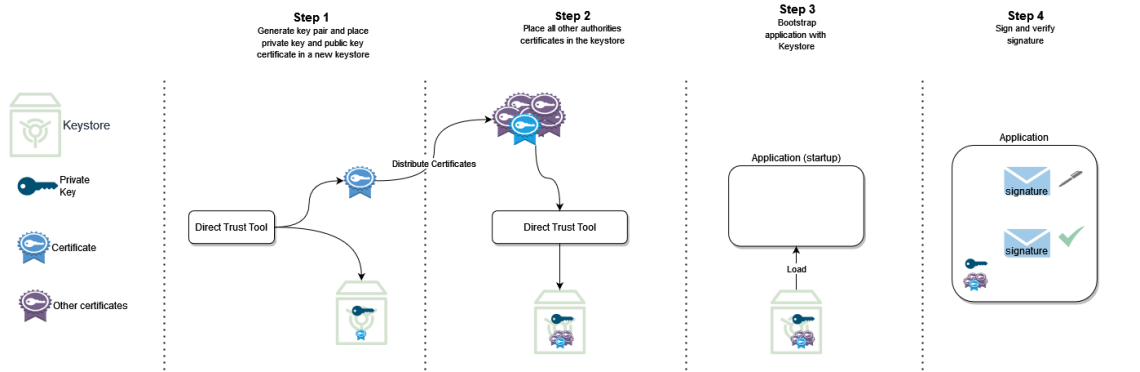


Fig. 1: The control components generate a certificate, share it out of band with the auditors, who import them in their keystore, thus enabling them to directly verify the authenticity of signed messages.

The certificate ceremony allows the auditors to verify that the data it received originated from the expected party.

2.4 Streamable Symmetric Encryption and Decryption for Dataset Security

The verifier's datasets are symmetrically encrypted as an additional security mechanism. Given the potentially huge size of the verifier's dataset, the verifier requires a decryption mechanism that can handle datasets too large to fit entirely in memory. This is an additional security layer since the data for the verifier (for instance the votes) are already encrypted. We refer to the system specification section **Streamable Symmetric Encryption and Decryption for Dataset Security** for the details on how to decrypt the verifier datasets.

2.5 Manual Checks by the Auditors

Certain domain-specific verifications cannot be run by software; they must be checked manually by the auditors operating the verifier. The explanatory report highlights that certain manual checks are necessary [2]:

[Explanatory Report, Sec 4.2.1]: The auditors must ascertain that the number of authentication credentials corresponds to the (official) number of authorised voters.

The verifier must create a document that can be easily understood and signed by auditors. This document should clearly indicate the datasets being used for the audit by printing the hash value of the datasets.

We assume that the auditors have access to the necessary information regarding the expected, correct configuration of the election event. This includes knowledge of the following parameters:

Parameter	Comment
Name and date of the election event	Public information
Number of elections and votes	Public information. In this context, the term ‘vote’ refers specifically to a group of referendum-style questions.
Number of productive and test ballot boxes	The number of ballot boxes relates to the number of counting circles in a canton and is known to the auditors
Number of eligible real and test voters	Can be learned from the electoral roll

Tab. 2: Overview of the auditor’s knowledge of the election event parameters

The auditors can learn the information in table 2 from publicly available sources, compare it against data from previous election events, or confirm it against the actual electoral roll.

The verifier will present a document to the auditors, displaying the values discussed above, which are sourced from the canton’s Election Event Configuration, as described in section 3.1. Before presenting the document, the verifier performs authenticity and consistency checks on the canton’s Election Event Configuration in sections 3.2 and 3.3 to ensure that the information is authentic and consistent. Only if all relevant verifications are successful, and the auditors have manually verified the information, will the auditors sign the document to confirm its accuracy.

The Election Event Configuration—signed by the canton (see section 3.2)—contains additional information about the election event, such as the precise wording of the questions, the names of the candidates and lists, and an identifier for each voting option. While the auditors can assume that this additional information is correct, it is still recommended that they manually verify the accuracy of this information by cross-checking it against other available sources.

The below pseudocode indicates the checks that the auditors must perform manually.

Verification 0.01 ManualChecksByAuditors

Context:

The protocol participants' certificates—received via an out-of-band channel (section 2.3).
When executing **VerifyTally**: the context dataset's hash value from the **VerifyConfigPhase**.
The auditors' knowledge of the election event parameters indicated in table 2.

Input:

A verifier's execution in the **VerifyConfigPhase** or **VerifyTally**.

The verifier's report(s) indicating:

- the certificate fingerprints
- the name and date of the election event
- the number of elections and votes
- the number of productive and test ballot boxes
- the number of eligible real and test voters
- the execution status of the **VerifyConfigPhase** or **VerifyTally** verifications

Operation:

- 1: When executing **VerifyTally**, check that the hash of the context dataset equals the value recorded during **VerifyConfigPhase**, and that the hash of the verifier's eCH-0222 XML equals the hash of the eCH-0222 XML imported into the system for consolidation and publication.
- 2: Check that the certificates' fingerprints match the expected ones.
- 3: Check that the name and date of the election event correspond to the expected ones.
- 4: Check that the number of elections and votes corresponds to the expected one.
- 5: Check that the number of productive and test ballot boxes corresponds to the expected one.
- 6: Check that the number of real and test voters corresponds to the expected one.
- 7: Check that all expected verifications executed successfully.

▷ See crypto primitives specification

Output:

⊤ if the verification succeeds, ⊥ otherwise.

In the **VerifyTally** phase, the auditors must have access to the signed audit document from the **VerifyConfigPhase**.

Once **VerifyTally** has been successfully executed, the auditors must review and acknowledge the election results. This process involves spot checking the results to ensure their plausibility and accuracy.

Besides the check in **ManualChecksByAuditors** that the correct version of eCH-0222 XML file is used, the auditors should record selected election results. These records can later be compared against the officially published results to detect any discrepancies.

2.6 Election Event Context

We refer the reader to the [system specification](#), which explains the election event context and that some algorithms are executed per election event, per verification card set, per ballot box, or voting card.

2.7 Basic Data Types

The [crypto primitives specification](#) details how we represent, convert, and operate on basic data types such as bytes, integers, strings, and arrays.

3 Setup Verification - VerifyConfigPhase

This section defines the verifications that need to be performed before the voters may be allowed to start voting. This is meant to ensure that the election event has been configured correctly and according to the specification, with all parties having provided the required proof for the data they generated. In case an error appears during verification of the setup phase, it must be determined if the cause is a misconfiguration of the verifier or if further investigation is needed. In the first case, the verifier may simply be configured and run anew. In the second case, once the root cause has been identified and fixed, the setup phase must be restarted from scratch.

3.1 Setup - Completeness

The VerifyConfigPhase requires a complete context (table 3). The implementation must ensure that the dataset contains the expected files in the correct file structure.

The placeholders represent the different identifiers present in the paths. To be more specific, the placeholder $\{j\}$ represents the control component index, the placeholder $\{vcs\}$ represents the verification card set identifier. The different consistency verifications ensure that these indices and identifiers are consistent to the information in the Election Event Context.

Description	Path
Election Event Context	context/electionEventContextPayload.json
Election Event Configuration	context/configuration-anonymized.xml
Online Control Component Public Keys	context/controlComponentPublicKeysPayload. $\{j\}$.json
Setup Component Public Keys	context/setupComponentPublicKeysPayload.json
Setup Component Tally Data	context/verificationCardSets/ $\{vcs\}$ /setupComponentTallyDataPayload.json

Tab. 3: The contents of the context dataset. The context dataset is used both in VerifyConfigPhase and VerifyTally

Verification 1.01 VerifySetupCompleteness

Input:

The context dataset.

Operation:

- 1: Verify that the dataset contains all required elements from table 3.
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

3.2 Setup - Authenticity

Table 4 provides an overview of the authenticity checks for the setup verification. Each element corresponds to an entry in table 3, and provides the details of what should be given as input to the `VerifySignature` algorithm.

Message Name	Signer	Message Content	Context Data
ElectionEventContext	Setup Comp.	ElectionEventContextPayload, see system specification table 19	(“election event context”, ee)
CantonConfig	Canton	configuration XML, see system specification table 16	-
ControlComponentPublicKeys	Online CC_j	ControlComponentPublicKeysPayload, see system specification table 19	(“OnlineCC keys”, j , ee)
SetupComponentPublicKeys	Setup Comp.	SetupComponentPublicKeysPayload, see system specification table 19	(“public keys”, “setup”, ee)
SetupComponentTallyData	Setup Comp.	SetupComponentTallyDataPayload, see system specification table 19	(“tally data”, ee, vcs)

Tab. 4: Overview of the authenticity checks for the setup verification

The configuration XML is signed with the algorithm `GenXMLSignature` as described in the section `Channel Security` of the system specification [11].

Verification 2.01 `VerifySignatureCantonConfig`

Context:

The trust store containing the system’s certificates.

Input:

Signed configuration XML from table 4.

Operation:

- 1: `VerifyXMLSignature(configuration XML,  $pk_{\text{canton}}$ )` ▷ see system specification. The canton’s public key pk_{canton} is extracted from the trust store.
-

Output:

$\top$ if verification succeeds, $\perp$ otherwise.

Verification 2.02 VerifySignatureSetupComponentPublicKeys

Context:

The trust store containing the system's certificates

Input:

The message SetupComponentPublicKeys from table 4

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: **for** $j \in [1, 4]$ **do**
 - 2: $h_{pk,j} \leftarrow (j, \mathbf{pk}_{CCR,j}, \pi_{pkCCR,j}, \mathbf{EL}_{pk,j}, \pi_{\mathbf{EL}pk,j})$
 - 3: **end for**
 - 4: $h_{pk} \leftarrow (h_{pk,1}, h_{pk,2}, h_{pk,3}, h_{pk,4})$
 - 5: $\mathbf{hPublicKeys} \leftarrow (h_{pk}, \mathbf{EB}_{pk}, \pi_{\mathbf{EB}}, \mathbf{EL}_{pk}, \mathbf{pk}_{CCR})$
 - 6: $h \leftarrow ((p, q, g), \mathbf{ee}, \mathbf{hPublicKeys})$
 - 7: $\text{VerifySignature}(\text{"sdm_config"}, h, (\text{"public keys"}, \text{"setup"}, \mathbf{ee}), s)$
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 2.02 are provided in

[verify-signature-setup-component-public-keys.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

Verification 2.03 VerifySignatureControlComponentPublicKeys

Context:

The trust store containing the system's certificates

Input:

The message ControlComponentPublicKeys from table 4

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: $\mathbf{hPublicKeys} \leftarrow (j, \mathbf{pk}_{CCR,j}, \pi_{pkCCR,j}, \mathbf{EL}_{pk,j}, \pi_{\mathbf{EL}pk,j})$
 - 2: $h \leftarrow ((p, q, g), \mathbf{ee}, \mathbf{hPublicKeys})$
 - 3: $\text{VerifySignature}(\text{"control_component_j"}, h, (\text{"OnlineCC keys"}, j, \mathbf{ee}), s)$
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 2.03 are provided in

[verify-signature-control-component-public-keys.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

Verification 2.04 VerifySignatureSetupComponentTallyData

Context:

The trust store containing the system's certificates

Input:

The message SetupComponentTallyData from table 4

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: $h \leftarrow ((p, q, g), ee, vcs, vc, ballotBoxDefaultTitle, K)$
 - 2: $\text{VerifySignature}(\text{"sdm_config"}, h, (\text{"tally data"}, ee, vcs), s)$
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 2.04 are provided in

[verify-signature-setup-component-tally-data.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

Verification 2.05 VerifySignatureElectionEventContext

Context:

The trust store containing the system's certificates

Input:

The message ElectionEventContext from table 4

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: **for** $vcs \in vcs$ **do**
 - 2: $h_{pTable,j} \leftarrow \left(\left((v_0, \tilde{p}_0, \sigma_0, \tau_0), \dots, (v_{n-1}, \tilde{p}_{n-1}, \sigma_{n-1}, \tau_{n-1}) \right) \right)$
 - 3: $h_{vcs,j} \leftarrow (vcs, vcsAlias, vcsDesc, bb, t_{s,bb}, t_{f,bb}, testBallotBox,$
 - 4: $h_{\Lambda,j} \leftarrow ((\psi_0, \dots, \psi_{t-1}), (DoI_0, \dots, DoI_{k-1}), (pg_0, \dots, pg_{t-1}), (ag_0, \dots, ag_{\eta-1}))$
 - 5: $h_{vcs,j} \leftarrow (vcs, vcsAlias, vcsDesc, bb, t_{s,bb}, t_{f,bb}, testBallotBox, N_E, gracePeriod, h_{pTable,j}, h_{\Lambda,j})$
 - 6: **end for**
 - 7: $h_{vcs} \leftarrow (h_{vcs,0}, h_{vcs,1}, \dots, h_{vcs,N_{bb}-1})$
 - 8: $hContext \leftarrow ((p, q, g), ee, eeAlias, eeDesc, h_{vcs}, t_{s,ee}, t_{f,ee}, n_{max}, \psi_{max}, \delta_{max})$
 - 9: $h \leftarrow ((p, q, g), seed, p, hContext, tenantId)$
 - 10: $\text{VerifySignature}(\text{"sdm_config"}, h, (\text{"election event context"}, ee), s)$
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 2.05 are provided in

[verify-signature-election-event-context.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

3.3 Setup - Consistency

For setup - consistency we have the following verifications.

Verification 3.01 VerifyEncryptionGroupConsistency

Input:

The encryption group parameters included in the following files from table 3:

- Election Event Context
 - Online Control Components Public Keys ▷ 1 per component
 - Setup Component Tally Data ▷ 1 per verification card set
-

Operation:

Verify that the encryption group parameters are consistent across all files.

Output:

⊤ if all encryption group parameters are identical, ⊥ otherwise.

Verification 3.02 VerifyNodeIdsConsistency

Input:

The node IDs included in the following file from table 3:

- Online Control Component Public Keys ▷ 1 per component
-

Operation:

- 1: Verify that the node IDs are consistent across all files, i.e. that all files have exactly one contribution from each node.
-

Output:

⊤ if the node IDs are consistent, ⊥ otherwise.

Verification 3.03 VerifyFileNameNodeIdsConsistency

Input:

The following file from the context dataset in table 3:

- Online Control Component Public Keys ▷ 1 per component
-

Operation:

Verify that the file names match the paths in the directory of the dataset.

Output:

⊤ if the file names are consistent, ⊥ otherwise.

Verification 3.04 VerifyElectionEventIdConsistency

Input:

The election event ID included in the files from table 3.

Operation:

- 1: Verify that the election event ID **ee** is consistent across all files.
-

Output:

$\top$ if the election event ID is consistent, $\perp$ otherwise.

Verification 3.05 VerifyVerificationCardSetIdsConsistency

Input:

The verification card set IDs included in the files from table 3.

Operation:

- 1: Verify that the verification card set IDs **vcs** are consistent across all files.
 - 2: Verify that the path names containing the verification card set ID in the context dataset match the verification card set ID within the files.
-

Output:

$\top$ if the verification card set IDs are consistent, $\perp$ otherwise.

Verification 3.06 VerifyFileNameVerificationCardSetIdsConsistency

Input:

The Election Event Context from table 3.
The paths of the context dataset (table 3).

Operation:

- 1: Verify that path names of the context dataset match the list of verification card set IDs in the Election Event Context.
-

Output:

$\top$ if the verification card set IDs between the dataset and the Election Event Context are consistent, $\perp$ otherwise.

Verification 3.07 VerifyVerificationCardIdsConsistency

Input:

The following files from table 3:

- Election Event Context
- Setup Component Tally Data

Operation:

- 1: Verify that there are no duplicate verification card IDs $\mathbf{vc}_{id}$ across all files.
- 2: Verify that the number of verification card IDs matches the number of eligible voters for all verification card sets.

Output:

$\top$ if all verification card IDs are consistent, $\perp$ otherwise.

Verification 3.08 VerifyCCRChoiceReturnCodesPublicKeyConsistency

Input:

The CCR Choice Return Codes encryption public keys ($\mathbf{pk}_{CCR_j}$) included in the following files from table 3:

- Online Control Component Public Keys $\triangleright$ 1 per component
- Setup Component Public Keys

Operation:

- 1: **for** $j \in [1, 4]$ **do**
- 2: $ok_j \leftarrow$ the CCR Choice Return Codes encryption public keys for control component j are identical from both sources
- 3: **end for**

Output:

$\top$ if all keys are identical, $\perp$ otherwise.

Verification 3.09 VerifyCCMElectionPublicKeyConsistency

Input:

The CCM election public keys ($[..^1]EL_{pk,j}$) included in the following files from table 3:

- Online Control Component Public Keys $\triangleright$ 1 per component
- Setup Component Public Keys

Operation:

- 1: **for** $j \in [1, 4]$ **do**
- 2: $ok_j \leftarrow$ the CCM election public key for control component j is identical from both sources
- 3: **end for**

Output:

$\top$ if all keys are identical, $\perp$ otherwise.

Verification 3.10 VerifyCCMAndCCRSchnorrProofsConsistency

Input:

- Online Control Component Public Keys $\triangleright$ 1 per component
 - Setup Component Public Keys
-

Operation:

- 1: Verify that the CCM and CCR Schnorr proofs are identical from both sources.
-

Output:

$\top$ if all [. .²] Schnorr proofs are identical, $\perp$ otherwise.

Verification 3.11 VerifyChoiceReturnCodesPublicKeyConsistency

Input:

- Online Control Component Public Keys $\triangleright$ 1 per component
 - Setup Component Public Keys
-

Operation:

- 1: Verify that $\mathbf{pk}_{\text{CCR}} = \prod_{j=1}^4 \mathbf{pk}_{\text{CCR}_j} \pmod{p}$ $\triangleright$ $\mathbf{pk}_{\text{CCR}_j}$ taken from the Online Control Component Public Keys
-

Output:

$\top$ if the Choice Return Codes encryption public key $\mathbf{pk}_{\text{CCR}}$ was correctly combined, $\perp$ otherwise.

Verification 3.12 VerifyElectionPublicKeyConsistency

Input:

- Online Control Component Public Keys $\triangleright$ 1 per component
 - Setup Component Public Keys
-

Operation:

- 1: Verify that $\mathbf{EL}_{\mathbf{pk}} = \mathbf{EB}_{\mathbf{pk}} \cdot \prod_{j=1}^4 \mathbf{EL}_{\mathbf{pk},j} \pmod{p}$ $\triangleright$ $\mathbf{EL}_{\mathbf{pk},j}$ taken from the Online Control Component Public Keys
-

Output:

$\top$ if the election public key $\mathbf{EL}_{\mathbf{pk}}$ was correctly combined, $\perp$ otherwise.

Verification 3.13 must derive the expected voting options from the **configuration XML**. See table 3 (Election Event Configuration). The **configuration XML** describes either elections or referendum-style votes. Table 5 describes how to derive the actual voting options and semantic information, depending on which kind of election or referendum-style vote is described in the **configuration XML**. Also the correctness information can be constructed. See section **Encoding of Voting Options** of the system specification for how to construct the correctness information. [We normalize the semantic information for abstention voting options since the presentation group position might differ between verification card sets.](#)

Type	Actual Voting Option	Normalized Semantic Information
vote / Popular Vote Answers	question identifier answer identifier	“NON_BLANK” or “BLANK” question answer
election/ Lists	election identifier list identifier	“NON_BLANK” or “BLANK” list description
election/ Candidates	election identifier candidate identifier candidate accumulation	“NON_BLANK” family name call name date/year of birth
election / Blank candidate position	election identifier blank candidate position identifier	“BLANK” EMPTY_CANDIDATE_POSITION-[position number]
election / Write-in positions	election identifier write-in position identifier	“WRITE_IN” WRITE_IN_POSITION-[position number]
Abstention positions	presentation group identifier abstention position identifier	“ABSTENTION” [.. ³][.. ⁴][.. ⁵]ABSTENTION_POSITION-[position number]

Tab. 5: Overview of the different voting option types necessary to validate the pTable.

Verification 3.13 VerifyPrimesMappingTableConsistency

Input:

- | | |
|--|---------------|
| Election Event Context | ▷ See table 3 |
| Election Event Configuration configuration XML | ▷ See table 3 |
-

Operation:

- 1: Verify that the same actual voting option v_i maps to the same encoded voting option $\tilde{p}_i$, that the same actual voting option v_i maps to the same [normalized](#) semantic information σ_i and that the same actual voting option v_i maps to the same correctness information τ_i in all verification card sets.
 - 2: Verify that the actual voting options, [normalized](#) semantic information and correctness information in the pTable correspond to the **configuration XML**, see table 5.
 - 3: Verify that the number of entries in the pTable correspond to the **configuration XML** taking into account possible accumulation of candidates.
-

Output:

⊤ if the primes mapping tables pTable in all verification card sets are consistent and correspond to the **configuration XML**, ⊥ otherwise.

Verification 3.14 VerifyTotalVotersConsistency

Input:

The following files from table 3:

- Election Event Configuration configuration XML
- Election Event Context

Operation:

- 1: Verify that the number of voters in the Election Event Configuration matches the sum of the number of eligible voters in all verification card sets in the Election Event Context.

Output:

$\top$ if the number of voters is consistent, $\perp$ otherwise.

3.4 Setup - Integrity

All pseudocode algorithms define the domain for each input. All inputs must be verified to be in the expected domains.

3.5 Setup - Evidence

The pseudocode algorithms in this section verify the evidence generated during the setup phase. Namely, these elements demonstrate that the Swiss Post Voting System configured cryptographically sound parameters, associated each voting option to a prime number, generated the correct number of voting cards, and signed the relevant configuration information. A positive verification result of these elements indicates to the auditor that the configuration allows the system to perform reasonably secure operations and to provide meaningful cryptographic evidence. In contrast, failed verifications in `VerifyConfigPhase` cast doubts about the system's security properties: secure encryption of votes, unforgeability of digital signatures, and the soundness of zero-knowledge proofs rely on the proper configuration of the cryptographic parameters.

The **seed** value which is an input to Verification 5.01 must be of the correct format as defined in the system specification [11], in the section **Cryptographic Parameters and System Security Level**.

Verification 5.01 VerifyEncryptionParameters

Input: $\triangleright$ All inputs are taken from table 3 – Election Event Context

Provided group modulus $\hat{p} \in \mathbb{P}$

Provided group cardinality $\hat{q} \in \mathbb{P}$ s.t. $\hat{p} = 2\hat{q} + 1$

Provided group generator $\hat{g} \in \mathbb{G}_q$

seed $\in \mathbb{A}_{UCS}^{16}$

$\triangleright$ The name of the election event in the format specified in the system specification

Require:

Verify that $|\hat{p}|$ corresponds to the *standard* security level $\triangleright$ See the section *Security Level* in the crypto primitives specification

Operation:

- 1: $(p, q, g) \leftarrow \text{GetEncryptionParameters}(\text{seed})$ $\triangleright$ See crypto primitives specification
 - 2: **if** $(p = \hat{p}) \wedge (q = \hat{q}) \wedge (g = \hat{g})$ **then**
 - 3: **return** $\top$
 - 4: **else**
 - 5: **return** $\perp$
 - 6: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

The verifier checks that the small primes—excluding the generator g —of a mathematical group $\mathbb{G}_q$ encode the voting options. Since the voting client multiplies all selected voting options prior to encryption, the verifier ensures that the maximum product of ψ_{sup} voting options does not exceed p to prevent modulo overflow.

Verification 5.02 VerifySmallPrimeGroupMembers

Context: $\triangleright$ All contexts are taken from table 3 – Election Event Context

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$

Group generator $g \in \mathbb{G}_q$

Maximum supported number of voting options $n_{\text{sup}} \in \mathbb{N}^+$

Input:

The small prime group members in ascending order $\mathbf{p} = (p_0, \dots, p_{n_{\text{sup}}-1}) \in (\mathbb{G}_q \cap \mathbb{P}) \setminus \{2, 3\}, (p_i < p_{i+1}) \forall i \in [0, n_{\text{sup}} - 1)$

Operation:

- 1: $\mathbf{p}' \leftarrow \text{GetSmallPrimeGroupMembers}(p, q, g, n_{\text{sup}})$ $\triangleright$ See crypto primitives specification
 - 2: **if** $\mathbf{p}' = \mathbf{p}$ **then**
 - 3: **return** $\top$
 - 4: **else**
 - 5: **return** $\perp$
 - 6: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

The **VerifyVotingOptions** algorithm follows **VerifySmallPrimeGroupMembers** and assumes that the latter algorithm verified the list of small prime group members. Hence, we label this input argument as a *trusted* input.

Moreover, the system specification [11] elaborates in the section **Electoral Model** that different voters might have different voting options.

Hence, the algorithm requires as input a sorted, consolidated list of encoded voting options across all voting card sets of size n_{total} ($n_{\text{total}} \geq n_{\text{max}}$).

Verification 5.03 VerifyVotingOptions

Context: $\triangleright$ All contexts are taken from table 3 – Election Event Context

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$

Group generator $g \in \mathbb{G}_q$

Maximum supported number of voting options $n_{\text{sup}} \in \mathbb{N}^+$

Maximum supported number of selections $\psi_{\text{sup}} \in \mathbb{N}^+$

Input:

Small prime group members in ascending order $\mathbf{p} = (p_0, \dots, p_{n_{\text{sup}}-1})$, $p_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus \{2, 3\}$, $(p_i < p_{i+1}) \forall i \in [0, n_{\text{sup}} - 1)$ $\triangleright$ *Trusted* input that was verified in verification 5.02

Encoded voting options in ascending order $\tilde{\mathbf{p}} = (\tilde{p}_0, \dots, \tilde{p}_{n_{\text{total}}-1})$, $\tilde{p}_i \in (\mathbb{G}_q \cap \mathbb{P}) \setminus \{2, 3\}$, $(\tilde{p}_i < \tilde{p}_{i+1}) \forall i \in [0, n_{\text{total}} - 1)$ $\triangleright$ The keys of the prime mapping tables in table 3 – Tally Data

Require:

$\psi_{\text{sup}} \leq n_{\text{sup}}$

$0 < n_{\text{total}} \leq n_{\text{sup}}$

Operation:

1: $\mathbf{p}' \leftarrow (p_0, \dots, p_{n_{\text{total}}-1})$

2: **if** $\mathbf{p}' = \tilde{\mathbf{p}}$ **then**

3: $\text{verifA} \leftarrow \top$

4: **else**

5: $\text{verifA} \leftarrow \perp$

6: **end if**

7: $\text{verifB} \leftarrow \prod_{i=(n_{\text{sup}}-\psi_{\text{sup}})}^{n_{\text{sup}}-1} p_i < p$ $\triangleright$ The product of the ψ_{sup} last primes (the largest possible encoded vote) must be smaller than p

8: **return** $\text{verifA} \wedge \text{verifB}$

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

The verifier checks the Schnorr proofs of knowledge that were generated during the configuration phase. This prevents a malicious party from providing a public key without knowing the corresponding secret key. This verification takes the Schnorr proofs from the setup component's public keys and assumes that the consistency verifications check that they correspond to the control component's data. The Schnorr proofs include the entire election event context as explained in the system specification. Thereby, it is ensured that the control components and the verifier agree on the election event context and in particular the list of ballot boxes and the number of voters that should be verified.

Verification 5.04 VerifySchnorrProofs

Context:

The Election Event Context ▷ Including $p, q, g, \mathbf{ee}, \psi_{\max}, \delta_{\max}$ ▷ See table 3 – Election Event Context

Input:

▷ All inputs are taken from table 3 – Setup Component Public Keys

CCR Choice Return Codes encryption keys $(\mathbf{pk}_{\text{CCR}_1}, \mathbf{pk}_{\text{CCR}_2}, \mathbf{pk}_{\text{CCR}_3}, \mathbf{pk}_{\text{CCR}_4}) \in (\mathbb{G}_q^{\psi_{\max}})^4$

CCR Schnorr proofs of knowledge $(\pi_{\text{pkCCR},1}, \pi_{\text{pkCCR},2}, \pi_{\text{pkCCR},3}, \pi_{\text{pkCCR},4}) \in ((\mathbb{Z}_q \times \mathbb{Z}_q)^{\psi_{\max}})^4$

CCM election public keys $(\text{EL}_{\text{pk},1}, \text{EL}_{\text{pk},2}, \text{EL}_{\text{pk},3}, \text{EL}_{\text{pk},4}) \in (\mathbb{G}_q^{\delta_{\max}})^4$

CCM Schnorr proofs of knowledge $(\pi_{\text{ELpk},1}, \pi_{\text{ELpk},2}, \pi_{\text{ELpk},3}, \pi_{\text{ELpk},4}) \in ((\mathbb{Z}_q \times \mathbb{Z}_q)^{\delta_{\max}})^4$

Electoral board public key $\text{EB}_{\text{pk}} \in \mathbb{G}_q^{\delta_{\max}}$

Electoral board Schnorr proofs of knowledge $\pi_{\text{EB}} \in (\mathbb{Z}_q \times \mathbb{Z}_q)^{\delta_{\max}}$

Require: $\delta_{\max} - 1 \leq \psi_{\max}$

Operation:

- 1: Prepare the *Input* containing all public keys and Schnorr proofs
 - 2: $\text{VerifSchnorrKeyGeneration} \leftarrow \text{VerifyKeyGenerationSchnorrProofs}(\text{Input})$ ▷ see system specification
 - 3: **if** $\text{VerifSchnorrKeyGeneration}$ **then**
 - 4: **return** $\top$
 - 5: **else**
 - 6: **return** $\perp$
 - 7: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

4 Final Verification - VerifyTally

This section presents the verifications needed to ascertain the final tally reflects the choices made by the voters. Since this is performed after the voting period has closed, typically slightly over a month after the first verification run, the verifier verifies the tally phase using the context dataset obtained in the `VerifyConfigPhase`.

`VerifyTally` succeeds *only* if the validation of *all* ballot boxes succeed. Since every control component in the tally phase verifies the output of the preceding control components, failed verifications are unlikely to happen during `VerifyTally`. The most probable failure scenario would be a failed verification of the Tally control component's output—since no control component verifies these operations. However, in that case, one could easily re-run the Tally control component and re-verify the results.

4.1 Final - Completeness

`VerifyTally` requires both the context (table 3) and the tally dataset (table 6).

The placeholders represent the different identifiers present in the paths. To be more specific, the placeholder `#{j}` represents the control component index and the placeholder `#{bb}` represents the ballot box identifier. The different consistency verifications ensure that these indices and identifiers are consistent to the information in the Election Event Context.

Description	Path
Control Component Ballot Box	<code>tally/ballotBoxes/#{bb}/controlComponentBallotBoxPayload_#{j}.json</code>
Online Control Component Shuffle	<code>tally/ballotBoxes/#{bb}/controlComponentShufflePayload_#{j}.json</code>
Tally Control Component Shuffle	<code>tally/ballotBoxes/#{bb}/tallyComponentShufflePayload.json</code>
Tally Control Component Votes	<code>tally/ballotBoxes/#{bb}/tallyComponentVotesPayload.json</code>
Tally Control Component Detailed Results	<code>tally/eCH-0222*.xml</code>

Tab. 6: The contents of the tally dataset. The tally dataset is used only in `VerifyTally`

Verification 6.01 VerifyTallyCompleteness

Input:

The context and tally datasets.

Operation:

- 1: Verify that the datasets contain all required elements from table 3 and table 6.
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

4.2 Final - Authenticity

Table 7 provides an overview of the authenticity checks for the tally verification. Each element corresponds to an entry in table 6, and provides the details of what should be given as input to the `VerifySignature` algorithm.

Message Name	Signer	Message Content	Context Data
ControlComponentBallotBox	Online CC_j	ControlComponentBallotBoxPayload, see system specification table 19	(“ballotbox”, j , ee, bb)
ControlComponentShuffle	Online CC_j	ControlComponentShufflePayload, see system specification table 19	(“shuffle”, j , ee, bb)
TallyComponentShuffle	Tally CC	TallyComponentShufflePayload, see system specification table 19	(“shuffle”, “offline”, ee, bb)
TallyComponentVotes	Tally CC	TallyComponentVotesPayload, see system specification table 19	(“decoded votes”, ee, bb)
TallyComponentEch0222	Tally CC	eCH-0222 XML, see system specification table 18	-

Tab. 7: Overview of the authenticity checks for the final verification

The eCH-0222 XML is signed with the algorithm `GenXMLSignature` as described in the section `Channel Security` of the system specification [11].

Verification 7.01 `VerifySignatureControlComponentBallotBox`

Context:

The trust store containing the system’s certificates

Input:

The message `ControlComponentBallotBox` from table 7

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: **for** $i \in [0, N_E)$ **do**
 - 2: $hVotes_i \leftarrow ((ee, vcs_i, vc_{j,i}), E1_{j,i}, \widetilde{E1}_{j,i}, E2_{j,i}, \pi_{Exp,j,i}, \pi_{EqEnc,j,i})$
 - 3: **end for**
 - 4: $hVotes \leftarrow (hVotes_0, \dots, hVotes_{N_E-1})$
 - 5: $h \leftarrow ((p, q, g), ee, bb, j, hVotes)$
 - 6: `VerifySignature`(“control_component_” j ”, h , (“ballotbox”, j , ee, bb), s)
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 7.01 are provided in

[verify-signature-control-component-ballot-box.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

Verification 7.02 VerifySignatureControlComponentShuffle

Context:

The trust store containing the system's certificates

Input:

The message ControlComponentShuffle from table 7

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: $\text{hShuffle} \leftarrow (\mathbf{c}_{\text{mix},j}, \pi_{\text{mix},j})$
- 2: $\text{hDecryption} \leftarrow (\mathbf{c}_{\text{dec},j}, \pi_{\text{dec},j})$
- 3: $h \leftarrow ((p, q, g), \text{ee}, \text{bb}, \text{hShuffle}, \text{hDecryption})$
- 4: $\text{VerifySignature}(\text{"control_component_j"}, h, (\text{"shuffle"}, j, \text{ee}, \text{bb}), s)$

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 7.02 are provided in

[verify-signature-control-component-shuffle.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

Verification 7.03 VerifySignatureTallyComponentShuffle

Context:

The trust store containing the system's certificates

Input:

The message TallyComponentShuffle from table 7

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: $\text{hShuffle} \leftarrow (\mathbf{c}_{\text{mix},5}, \pi_{\text{mix},5})$
- 2: $\text{hDecryption} \leftarrow (\mathbf{m}, \pi_{\text{dec},5})$
- 3: $h \leftarrow (\text{ee}, \text{bb}, \text{hShuffle}, \text{hDecryption})$
- 4: $\text{VerifySignature}(\text{"sdm_tally"}, h, (\text{"shuffle"}, \text{"offline"}, \text{ee}, \text{bb}), s)$

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 7.03 are provided in

[verify-signature-tally-component-shuffle.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

Verification 7.04 VerifySignatureTallyComponentVotes

Context:

The trust store containing the system's certificates

Input:

The message TallyComponentVotes from table 7

The signature $s \in \mathcal{B}^*$

Operation: ▷ For all algorithms see the crypto primitives specification ▷ We use nested structures in this algorithm

- 1: $\mathbf{hVotes} \leftarrow ((\hat{p}_0, \dots, \hat{p}_{\psi-1})_0, \dots, (\hat{p}_0, \dots, \hat{p}_{\psi-1})_{N_c-1})$
 - 2: $\mathbf{hDecodedVotes} \leftarrow ((\hat{v}_0, \dots, \hat{v}_{\psi-1})_0, \dots, (\hat{v}_0, \dots, \hat{v}_{\psi-1})_{N_c-1})$
 - 3: $\mathbf{hWriteIns} \leftarrow ((\hat{s}_0, \dots, \hat{s}_{k-1})_0, \dots, (\hat{s}_0, \dots, \hat{s}_{k-1})_{N_c-1})$
 - 4: $h \leftarrow ((p, q, g), \mathbf{ee}, \mathbf{bb}, \mathbf{hVotes}, \mathbf{hDecodedVotes}, \mathbf{hWriteIns})$
 - 5: $\text{VerifySignature}(\text{"sdm_tally"}, h, (\text{"decoded votes"}, \mathbf{ee}, \mathbf{bb}), s)$
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

Test values for the verification 7.04 are provided in

[verify-signature-tally-component-votes.json](#).

The value $d = \text{Base64Encode}(\text{RecursiveHash}(h))$ is provided to help debugging.

Verification 7.05 VerifySignatureTallyComponentECH0222

Context:

The trust store containing the system's certificates

Input:

Signed eCH-0222 XML from table 7

Operation:

- 1: $\text{VerifyXMLSignature}(\text{eCH-0222 XML}, \mathbf{pk}_{\text{tallyComponent}})$ ▷ see system specification. The tally component's public key $\mathbf{pk}_{\text{tallyComponent}}$ is extracted from the trust store.
-

Output:

$\top$ if the verification succeeds, $\perp$ otherwise.

4.3 Final - Consistency

For final - consistency we have the following verifications.

Verification 8.01 VerifyEncryptionGroupConsistency

Input:

The Election Event Context - see table 3.

The files from table 6.

Operation:

Verify that the encryption group parameters are consistent across all files.

Output:

$\top$ if all encryption group parameters are identical, $\perp$ otherwise.

Verification 8.02 VerifyNodeIdsConsistency

Input:

The node IDs included in the following files from table 6:

- | | |
|------------------------------------|----------------------------------|
| - Control Component Ballot Box | $\triangleright$ 1 per component |
| - Online Control Component Shuffle | $\triangleright$ 1 per component |

Operation:

- 1: Verify that the node IDs are consistent across all files, i.e. that all files have exactly one contribution from each node.

Output:

$\top$ if the node IDs are consistent, $\perp$ otherwise.

Verification 8.03 VerifyFileNameNodeIdsConsistency

Input:

The following files from the tally dataset in table 6:

- | | |
|------------------------------------|----------------------------------|
| - Control Component Ballot Box | $\triangleright$ 1 per component |
| - Online Control Component Shuffle | $\triangleright$ 1 per component |

Operation:

Verify that the file names match the paths in the directory of the dataset.

Output:

$\top$ if the file names are consistent, $\perp$ otherwise.

Verification 8.04 VerifyElectionEventIdConsistency

Input:

- The election event ID from the Election Event Context - see table 3.
- The election event ID included in the files from table 6.

Operation:

- 1: Verify that the election event ID ee is consistent across all files.

Output:

$\top$ if the election event ID is consistent, $\perp$ otherwise.

Verification 8.05 VerifyBallotBoxIdsConsistency

Input:

- The ballot box IDs included in the files from table 6.

Operation:

- 1: Verify that the ballot box IDs are consistent across all files.
- 2: Verify that the path names containing the ballot box ID in the tally dataset match the ballot box ID within the files.

Output:

$\top$ if the ballot box IDs are consistent, $\perp$ otherwise.

Verification 8.06 VerifyFileNameBallotBoxIdsConsistency

Input:

- The Election Event Context from table 3.
- The paths of the tally dataset (table 6).

Operation:

- 1: Verify that path names of the tally dataset match the list of ballot box IDs in the Election Event Context.

Output:

$\top$ if the ballot box IDs between the tally dataset and the Election Event Context are consistent, $\perp$ otherwise.

Verification 8.07 VerifyVerificationCardIdsConsistency

Input:

- The verification card IDs from the Setup Component Tally Data from table 3.
- The Election Event Context from table 3 to map the verification card set ID to the ballot box ID.
- The verification card IDs from the Control Component Ballot Box from table 6. $\triangleright$ 1 per component

Operation:

- 1: Verify that the verification card IDs in the Control Component Ballot Boxes are a subset of the verification card IDs of the Setup Component Tally Data.

Output:

$\top$ if all ballot boxes contain the expected verification card IDs, $\perp$ otherwise.

Verification 8.08 VerifyConfirmedEncryptedVotesConsistency

Input:

- The following files from the tally dataset in table 6:
 - Control Component Ballot Box $\triangleright$ 1 per component

Operation:

- 1: Verify that the confirmed encrypted votes—including the verification card ID vc_{id} , the encrypted vote E1 , the exponentiated encrypted vote $\widetilde{\text{E1}}$, the encrypted partial Choice Return Codes E2 , and the zero-knowledge proofs π_{Exp} and π_{EqEnc} —are identical across all control components—order notwithstanding.

Output:

$\top$ if the confirmed encrypted votes are consistent, $\perp$ otherwise.

Verification 8.09 VerifyCiphertextsConsistency

Input:

- The Election Event Context from table 3.
- The following files from the tally dataset in table 6:
 - Control Component Ballot Box $\triangleright$ 1 per component
 - Online Control Component Shuffle $\triangleright$ 1 per component
 - Tally Control Component Shuffle

Operation:

- 1: For each ballot box and every file, verify that the number of ciphertext elements equals the number of allowed write-ins plus one.

Output:

$\top$ if the ciphertexts have the expected number of elements in all ballot boxes, $\perp$ otherwise.

Verification 8.10 VerifyPlaintextsConsistency

Input:

- The Election Event Context from table 3.
- The Tally Control Component Shuffle from table 6.

Operation:

- 1: For each ballot box, verify that the number of plaintext elements after decryption equals the number of allowed write-ins plus one.

Output:

$\top$ if the plaintexts have the expected number of elements in all ballot boxes, $\perp$ otherwise.

Verification 8.11 VerifyNumberConfirmedEncryptedVotesConsistency

Input:

- The following files from the tally dataset in table 6:
 - Control Component Ballot Box $\triangleright$ 1 per component
 - Online Control Component Shuffle $\triangleright$ 1 per component
 - Tally Control Component Shuffle
 - Tally Control Component Votes

Operation:

- 1: Verify that the number of confirmed votes N_C is identical in Control Component Ballot Box and Tally Control Component Votes.
- 2: Verify that the number of mixed votes including trivial encryptions $\hat{N}_C$ is identical in Online Control Component Shuffle and Tally Control Component Shuffle.
- 3: Verify that if $N_C < 2$ then $\hat{N}_C = N_C + 2$, otherwise that $\hat{N}_C = N_C$. $\triangleright$ If there are strictly less than 2 confirmed votes we add trivial encryptions to the ballot box. See algorithm `GetMixnetInitialCiphertexts` in the system specification

Output:

$\top$ if the number of confirmed votes is consistent, $\perp$ otherwise.

4.4 Final - Integrity

All integrity checks are performed as part of the domain definitions of the pseudocode for the verifications below.

4.5 Final - Evidence

After the tally phase, the verifier repeats the Tally control component's verification of the online control components and verifies the Tally control component's operations themselves. The verification of the online control component's verifications comprises the following:

- Verify the voting client's proofs in the algorithm **VerifyVotingClientProofs** (see system specification). This verification also ensures that the verifier works with the same primes mapping table **pTable** as the control components.
- Verify the online control components' shuffle and decryption proofs in the algorithm **VerifyMixDecOffline** (see system specification). To this end, the verifier invokes the **GetMixnetInitialCiphertexts** algorithm.

Moreover, the verifier must check the operations of the Tally control component:

- Verify the Tally control component's shuffle and decryption proofs.
- Verify the Tally control component's processing of the plaintexts.
- Verify the Tally control component's generation of the tally file.

Verification 10.01 VerifyOnlineControlComponents

Context:

Election event ID **ee** $\in (\mathbb{A}_{Base16})^{1ID}$

Vector of verification card set IDs **vcs** $= (vcs_0, \dots, vcs_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{1ID})^{N_{bb}}$

Vector of ballot box IDs **bb** $= (bb_0, \dots, bb_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{1ID})^{N_{bb}}$

Election Event Context

▷ See table 3

Setup Component Public Keys

▷ See table 3

Input:

First Control Component Ballot Boxes (**vc**₁, **E1**₁, **E1**₁, **E2**₁, **π**_{Exp,1}, **π**_{EqEnc,1}) for all **bb**_{*i*} ▷ See table 6

Online Control Component Shuffles $\{c_{mix,j}, \pi_{mix,j}, c_{dec,j}, \pi_{dec,j}\}_{j=1}^4$ for all **bb**_{*i*} ▷ See table 6

Setup Component Tally Data (**vc**, **K**) for all **vcs**_{*i*} ▷ See table 3

Operation:

- 1: **for** $i \in [0, N_{bb})$ **do**
 - 2: **Input**_{**bb**_{*i*}} $\leftarrow (\mathbf{vc}_1, \mathbf{E1}_1, \widetilde{\mathbf{E1}}_1, \mathbf{E2}_1, \boldsymbol{\pi}_{Exp,1}, \boldsymbol{\pi}_{EqEnc,1}, \{c_{mix,j}, \pi_{mix,j}, c_{dec,j}, \pi_{dec,j}\}_{j=1}^4, \mathbf{vc}, \mathbf{K})$
 - 3: **bbOnlineCCVerif**_{*i*} $\leftarrow \text{VerifyOnlineControlComponentsBallotBox}(\text{Input}_{\mathbf{bb}_i})$ ▷ See Algorithm 4.1
 - 4: **end for**
 - 5: **if** **bbOnlineCCVerif**_{*i*} $\forall i$ **then**
 - 6: **return** $\top$
 - 7: **else**
 - 8: **return** $\perp$
 - 9: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful for *all* ballot boxes, $\perp$ otherwise.

Algorithm 4.1 VerifyOnlineControlComponentsBallotBox

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{1\text{ID}}$
 Verification card set ID $\mathbf{vcs} \in (\mathbb{A}_{Base16})^{1\text{ID}}$
 Ballot box ID $\mathbf{bb} \in (\mathbb{A}_{Base16})^{1\text{ID}}$
 Number of eligible voters $\mathbf{N_E} \in \mathbb{N}^+$ ▷ see system specification
 Primes Mapping Table $\mathbf{pTable} \in (\mathcal{D}_V \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times \mathbb{A}_{UCS}^* \times \mathcal{D}_\tau)^n$ ▷ ψ and δ can be derived from $\mathbf{pTable}$, see system specification
 Electoral Model Context $\Lambda = (\psi, \mathbf{DoI}, \mathbf{pg}, \mathbf{ag}) \in \left((\mathbb{N}^+)^t, (\mathcal{T}_1^{50})^k, [0, \eta]^t, \mathbb{Z}_2^\eta \right)$ ▷ see system specification
 Election public key $\mathbf{ELpk} = (\mathbf{ELpk}_{0,0}, \dots, \mathbf{ELpk}_{\delta_{\max}-1}) \in \mathbb{G}_q^{\delta_{\max}}$
 CCM election public keys $(\mathbf{ELpk}_{1,1}, \mathbf{ELpk}_{1,2}, \mathbf{ELpk}_{1,3}, \mathbf{ELpk}_{1,4}) \in (\mathbb{G}_q^{\delta_{\max}})^4$
 Electoral board public key $\mathbf{EBpk} \in \mathbb{G}_q^{\delta_{\max}}$
 Choice Return Codes encryption public key $\mathbf{pk}_{CCR} \in \mathbb{G}_q^{\psi_{\max}}$

Input:

Control component's list of confirmed verification card IDs $\mathbf{vc}_1 = (\mathbf{vc}_{1,0}, \dots, \mathbf{vc}_{1,\mathbf{N_C}-1}) \in \left((\mathbb{A}_{Base16})^{1\text{ID}} \right)^{\mathbf{N_C}}$
 Control component's list of encrypted, confirmed votes $\mathbf{E1}_1 = (\mathbf{E1}_{1,0}, \dots, \mathbf{E1}_{1,\mathbf{N_C}-1}) \in \left(\mathbb{G}_q^{\delta+1} \right)^{\mathbf{N_C}}$
 Control component's list of exponentiated, encrypted, confirmed votes $\widetilde{\mathbf{E1}}_1 = (\widetilde{\mathbf{E1}}_{1,0}, \dots, \widetilde{\mathbf{E1}}_{1,\mathbf{N_C}-1}) \in \left(\mathbb{G}_q^2 \right)^{\mathbf{N_C}}$
 Control component's list of encrypted, partial Choice Return Codes $\mathbf{E2}_1 = (\mathbf{E2}_{1,0}, \dots, \mathbf{E2}_{1,\mathbf{N_C}-1}) \in \left(\mathbb{G}_q^{\psi+1} \right)^{\mathbf{N_C}}$
 Control component's list of exponentiation proofs $\pi_{\text{Exp},1} = (\pi_{\text{Exp},1,0}, \dots, \pi_{\text{Exp},1,\mathbf{N_C}-1}) \in \left(\mathbb{Z}_q \times \mathbb{Z}_q \right)^{\mathbf{N_C}}$
 Control component's list of plaintext equality proofs $\pi_{\text{EqEnc},1} = (\pi_{\text{EqEnc},1,0}, \dots, \pi_{\text{EqEnc},1,\mathbf{N_C}-1}) \in \left(\mathbb{Z}_q \times \mathbb{Z}_q^2 \right)^{\mathbf{N_C}}$
 Preceding shuffled votes $\{\mathbf{c}_{\text{mix},j}\}_{j=1}^4 \in \left((\mathbb{G}_q^{\delta+1})^{\hat{\mathbf{N_C}}} \right)^4$
 Preceding shuffle proofs $\{\pi_{\text{mix},j}\}_{j=1}^4$ ▷ See the domain of the shuffle argument in the crypto primitives specification
 Preceding partially decrypted votes $\{\mathbf{c}_{\text{dec},j}\}_{j=1}^4 \in \left((\mathbb{G}_q^{\delta+1})^{\hat{\mathbf{N_C}}} \right)^4$
 Preceding decryption proofs $\{\pi_{\text{dec},j}\}_{j=1}^4 \in \left((\mathbb{Z}_q^{\delta+1})^{\hat{\mathbf{N_C}}} \right)^4$
 Vector of verification card IDs $\mathbf{vc} = (\mathbf{vc}_0, \dots, \mathbf{vc}_{\mathbf{N_E}-1}) \in ((\mathbb{A}_{Base16})^{1\text{ID}})^{\mathbf{N_E}}$
 Verification card public keys $\mathbf{K} = (K_0, \dots, K_{\mathbf{N_E}-1}) \in (\mathbb{G}_q)^{\mathbf{N_E}}$

Require: $\mathbf{N_E} \geq \mathbf{N_C}$

Require: $\hat{\mathbf{N_C}} = \mathbf{N_C}$ if $\mathbf{N_C} \geq 2$, otherwise $\hat{\mathbf{N_C}} = \mathbf{N_C} + 2$

▷ The algorithm runs with at least two votes

Require: $\mathbf{vc}_{1,i} \neq \mathbf{vc}_{1,k}, \forall i, k \in \{0, \dots, (\mathbf{N_C} - 1)\} \wedge i \neq k$

▷ All verification card IDs must be distinct

Operation:

1: if $\mathbf{N_C} \geq 1$ then ▷ Verifying the voting client proofs requires at least one confirmed vote
 2: $\mathbf{KMap} \leftarrow \left((\mathbf{vc}_0, K_0), \dots, (\mathbf{vc}_{\mathbf{N_E}-1}, K_{\mathbf{N_E}-1}) \right)$
 3: $\text{vcProofsVerif} \leftarrow \text{VerifyVotingClientProofs}(\mathbf{vc}_1, \mathbf{E1}_1, \widetilde{\mathbf{E1}}_1, \mathbf{E2}_1, \pi_{\text{Exp},1}, \pi_{\text{EqEnc},1}, \mathbf{KMap})$ ▷ see system specification
 4: else
 5: $\text{vcProofsVerif} \leftarrow \top$
 6: end if
 7: $\mathbf{vcMap}_1 \leftarrow \left((\mathbf{vc}_{1,0}, \mathbf{E1}_{1,0}), \dots, (\mathbf{vc}_{1,\mathbf{N_C}-1}, \mathbf{E1}_{1,\mathbf{N_C}-1}) \right)$
 8: $\mathbf{c}_{\text{init},1} \leftarrow \text{GetMixnetInitialCiphertexts}(\mathbf{vcMap}_1)$ ▷ see system specification
 9: $\text{shuffleProofsVerif} \leftarrow \text{VerifyMixDecOffline}(\mathbf{c}_{\text{init},1}, \{\mathbf{c}_{\text{mix},j}\}_{j=1}^4, \{\pi_{\text{mix},j}\}_{j=1}^4, \{\mathbf{c}_{\text{dec},j}\}_{j=1}^4, \{\pi_{\text{dec},j}\}_{j=1}^4)$ ▷ see system specification
 10: if $\text{vcProofsVerif} \wedge \text{shuffleProofsVerif}$ then
 11: return $\top$
 12: else
 13: return $\perp$
 14: end if

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box, $\perp$ otherwise.

Next, the verifier checks the Tally control component's operations.

Verification 10.02 VerifyTallyControlComponent

Context:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{1_{ID}}$

Vector of ballot box IDs $\mathbf{bb} = (\mathbf{bb}_0, \dots, \mathbf{bb}_{N_{bb}-1}) \in ((\mathbb{A}_{Base16})^{1_{ID}})^{N_{bb}}$

Election Event Context

▷ See table 3

Setup Component Public Keys

▷ See table 3

Input:

Last Online Control Component Shuffles $(\mathbf{c}_{mix,4}, \pi_{mix,4}, \mathbf{c}_{dec,4}, \boldsymbol{\pi}_{dec,4})$ for all $\mathbf{bb}_i$ ▷ See table 6

Tally Control Component Shuffles $(\mathbf{c}_{mix,5}, \pi_{mix,5}, \mathbf{m}, \boldsymbol{\pi}_{dec,5})$ for all $\mathbf{bb}_i$ ▷ See table 6

Tally Control Component Votes $(L_{votes}, L_{decodedVotes}, L_{writeIns})$ for all $\mathbf{bb}_i$ ▷ See table 6

Election Event Configuration configuration XML

▷ See table 3

Tally Control Component Detailed Results eCH-0222 XML

▷ See table 6

Key-value map of $L_{decodedVotes}$ per authorization name $\text{Map}_{decodedVotes}$

Key-value map of $L_{writeIns}$ per authorization name $\text{Map}_{writeIns}$

Operation:

```

1: for  $i \in [0, N_{bb})$  do
2:    $\text{Input}_{\mathbf{bb}_i} \leftarrow (\mathbf{c}_{dec,4}, \mathbf{c}_{mix,5}, \pi_{mix,5}, \mathbf{m}, \boldsymbol{\pi}_{dec,5}, L_{votes}, L_{decodedVotes}, L_{writeIns})$ 
3:    $\text{tallyVerif}_i \leftarrow \text{VerifyTallyControlComponentBallotBox}(\text{Input}_{\mathbf{bb}_i})$  ▷ See Algorithm 4.2
4: end for
5:  $\text{Input}_{eCH0222} \leftarrow (\text{configuration XML}, \text{eCH-0222 XML}, \text{Map}_{decodedVotes}, \text{Map}_{writeIns})$ 
6:  $\text{eCH0222Verif} \leftarrow \text{VerifyECH0222}(\text{Input}_{eCH0222})$  ▷ See Algorithm 4.4
7: if  $\text{tallyVerif}_i \forall i \wedge \text{eCH0222Verif}$  then
8:   return  $\top$ 
9: else
10:  return  $\perp$ 
11: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *all* ballot boxes, $\perp$ otherwise.

Algorithm 4.2 VerifyTallyControlComponentBallotBox

Context:

Group modulus $p \in \mathbb{P}$
 Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$
 Group generator $g \in \mathbb{G}_q$
 Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{1_{ID}}$
 Ballot box ID $\mathbf{bb} \in (\mathbb{A}_{Base16})^{1_{ID}}$
 Number of eligible voters $N_E \in \mathbb{N}^+$ $\triangleright$ see system specification
 Primes Mapping Table $\mathbf{pTable} \in (\mathcal{D}_v \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times \mathbb{A}_{UCS}^* \times \mathcal{D}_\tau)^n \triangleright \psi$ and δ can be derived from $\mathbf{pTable}$, see system specification
 Number of selections vector $\psi = (\psi_0, \dots, \psi_{t-1}) \in (\mathbb{N}^+)^t$
 Presentation groups $\mathbf{pg} = (\mathbf{pg}_0, \dots, \mathbf{pg}_{t-1}) \in [0, \eta)^t$
 Electoral board public key $\mathbf{EB}_{pk} \in \mathbb{G}_q^{\delta_{max}}$

Input:

The last online control component's partially decrypted votes $\mathbf{c}_{dec,4} \in (\mathbb{G}_q^{\delta+1})^{\hat{N}_C}$
 The tally component's shuffled votes $\mathbf{c}_{mix,5} \in (\mathbb{G}_q^{\delta+1})^{\hat{N}_C}$
 The tally component's shuffle proofs $\pi_{mix,5}$ $\triangleright$ See the domain of the shuffle argument in the crypto primitives specification
 The decrypted votes $\mathbf{m} = (m_0, \dots, m_{\hat{N}_C-1}) \in (\mathbb{G}_q^\delta)^{\hat{N}_C}$
 The decryption proofs $\pi_{dec,5} \in (\mathbb{Z}_q^{\delta+1})^{\hat{N}_C}$
 List of decrypted votes $L_{votes} = (\hat{\mathbf{p}}_0, \dots, \hat{\mathbf{p}}_{N_C-1}) \in (((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^\psi)^{N_C}$
 List of decoded votes $L_{decodedVotes} = (\hat{\mathbf{v}}_0, \dots, \hat{\mathbf{v}}_{N_C-1}) \in ((\mathcal{D}_v)^\psi)^{N_C}$
 List of decoded write-ins $L_{writeIns} = (\hat{\mathbf{s}}_0, \dots, \hat{\mathbf{s}}_{N_C-1}) \in ((\mathbb{A}_{latin}^*)^*)^{N_C}$

Require: $N_E \geq N_C$

Require: $\hat{N}_C = N_C$ if $N_C \geq 2$, otherwise $\hat{N}_C = N_C + 2$ $\triangleright$ The algorithm runs with at least two votes

Require: $\hat{\mathbf{p}}_i \subseteq \text{GetEncodedVotingOptions}(), \forall i \in \{0, \dots, (N_C - 1)\}$

Require: $\hat{\mathbf{p}}_{i,k} \neq \hat{\mathbf{p}}_{i,l}, \forall i \in \{0, \dots, (N_C - 1)\}, \forall k, l \in \{0, \dots, (\psi - 1)\} \wedge k \neq l$ $\triangleright$ A vote's selected encoded voting options must be distinct

Operation:

```

1:  $\mathbf{i}_{aux} \leftarrow (\mathbf{ee}, \mathbf{bb}, \text{"MixDecOffline"})$ 
2:  $\mathbf{EB}_{pk,cut} \leftarrow (\mathbf{EB}_{pk,0}, \dots, \mathbf{EB}_{pk,\delta-1})$ 
3:  $\text{shuffleVerif} \leftarrow \text{VerifyShuffle}(\mathbf{c}_{dec,4}, \mathbf{c}_{mix,5}, \pi_{mix,5}, \mathbf{EB}_{pk,cut})$   $\triangleright$  See crypto primitives specification
4: for  $i \in [0, \hat{N}_C)$  do
5:    $\text{decryptVerif}_i \leftarrow \text{VerifyDecryption}(\mathbf{c}_{mix,5,i}, \mathbf{EB}_{pk,cut}, m_i, \pi_{dec,5,i}, \mathbf{i}_{aux})$ 
6: end for  $\triangleright$  See crypto primitives specification
7:  $\text{processVerif} \leftarrow \text{VerifyProcessPlaintexts}(\mathbf{m}, L_{votes}, L_{decodedVotes}, L_{writeIns})$   $\triangleright$  See algorithm 4.3
8: if  $\text{shuffleVerif} \wedge \text{decryptVerif}_i \forall i \in [0, \hat{N}_C) \wedge \text{processVerif}$  then
9:   return  $\top$ 
10: else
11:   return  $\perp$ 
12: end if

```

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box, $\perp$ otherwise.

Algorithm 4.3 VerifyProcessPlaintexts

Context:

Group modulus $p \in \mathbb{P}$

Group cardinality $q \in \mathbb{P}$ s.t. $p = 2q + 1$

Group generator $g \in \mathbb{G}_q$

Primes Mapping Table $\mathbf{pTable} \in (\mathcal{D}_v \times ((\mathbb{G}_q \cap \mathbb{P}) \setminus g) \times \mathbb{A}_{UCS}^* \times \mathcal{D}_\tau)^n \triangleright \mathbf{pTable}$ is of the form $((v_0, \tilde{p}_0, \sigma_0, \tau_0), \dots, (v_{n-1}, \tilde{p}_{n-1}, \sigma_{n-1}, \tau_{n-1})) \triangleright \psi$ and δ can be derived from $\mathbf{pTable}$, see system specification

Number of selections vector $\psi = (\psi_0, \dots, \psi_{t-1}) \in (\mathbb{N}^+)^t$

Presentation groups $\mathbf{pg} = (\mathbf{pg}_0, \dots, \mathbf{pg}_{t-1}) \in [0, \eta]^t$

Input:

$\triangleright$ see system specification

List of plaintext votes $\mathbf{m} = (m_0, \dots, m_{\hat{N}_C-1}) \in (\mathbb{G}_q^\delta)^{\hat{N}_C}$

List of decrypted votes $L_{\text{votes}} = (\hat{\mathbf{p}}_0, \dots, \hat{\mathbf{p}}_{N_C-1}) \in (((\mathbb{G}_q \cap \mathbb{P}) \setminus g)^\psi)^{N_C} \triangleright$ We assume that the algorithm 4.2 verified that all votes' selected encoded voting options are distinct and a subset of the possible encoded voting options.

List of decoded votes $L_{\text{decodedVotes}} = (\hat{\mathbf{v}}_0, \dots, \hat{\mathbf{v}}_{N_C-1}) \in ((\mathcal{D}_v)^\psi)^{N_C}$

List of decoded write-ins $L_{\text{writeIns}} = (\hat{\mathbf{s}}_0, \dots, \hat{\mathbf{s}}_{N_C-1}) \in ((\mathbb{A}_{\text{latin}}^*)^*)^{N_C}$

Require: $\hat{N}_C \geq 2$

$\triangleright$ The algorithm runs with at least two votes

Require: $\hat{N}_C = N_C$ if $N_C \geq 2$, otherwise $\hat{N}_C = N_C + 2$

Operation:

- 1: $(L'_{\text{votes}}, L'_{\text{decodedVotes}}, L'_{\text{writeIns}}) \leftarrow \text{ProcessPlaintexts}(\mathbf{m}) \triangleright$ see system specification
 - 2: **if** $(L'_{\text{votes}} = L_{\text{votes}}) \wedge (L'_{\text{decodedVotes}} = L_{\text{decodedVotes}}) \wedge (L'_{\text{writeIns}} = L_{\text{writeIns}})$ **then**
 - 3: **return** $\top$
 - 4: **else**
 - 5: **return** $\perp$
 - 6: **end if**
-

Output:

The result of the verification: $\top$ if the verification is successful for *this specific* ballot box, $\perp$ otherwise.

In the following algorithm, the verifier regenerates the eCH-0222 XML and verifies its correctness. Due to the complexity of the XML file structure, we utilize **GenXMLSignature** to ensure that the tally control component and the verifiers eCH-0222 XML are semantically identical. **GenXMLSignature** canonicalizes and transforms the XML document, eliminating potential ambiguities such as OS-dependent line breaks and inconsistent whitespace. To achieve this, we generate a temporary secret key for signing, which is exclusively used within the **VerifyECH0222** algorithm. We assume a function **GenSecretKey** that generates a private signing key.

Algorithm 4.4 VerifyECH0222

Context:

Election event ID $\mathbf{ee} \in (\mathbb{A}_{Base16})^{1D}$

Input:

Election Event Configuration configuration XML ▷ See table 3
Tally Control Component Detailed Results eCH-0222 XML ▷ See table 6
Key-value map of $L_{\text{decodedVotes}}$ per authorization name $\text{Map}_{\text{decodedVotes}}$
Key-value map of L_{writeIns} per authorization name $\text{Map}_{\text{writeIns}}$

Operation:

```

1: eCH-0222 XML'  $\leftarrow$  CreateECH0222(configuration XML,  $\text{Map}_{\text{decodedVotes}}$ ,  $\text{Map}_{\text{writeIns}}$ ) ▷ see
   system specification
2: sk  $\leftarrow$  GenSecretKey() ▷ Create a temporary secret key for signing
3:  $D_{\text{signed}} \leftarrow \text{GenXMLSignature}(\text{eCH-0222 XML}, \text{sk})$  ▷ see system specification
4:  $[..^6]D'_{\text{signed}} \leftarrow \text{GenXMLSignature}(\text{eCH-0222 XML}', \text{sk})$ 
5: d  $\leftarrow \text{ExtractDigest}(D_{\text{signed}})$  ▷ see system specification
6: d'  $\leftarrow \text{ExtractDigest}(D'_{\text{signed}})$ 
7: if d = d' then
8:   return  $\top$ 
9: else
10:  return  $\perp$ 
11: end if

```

Output:

The result of the verification: $\top$ if the verification is successful, $\perp$ otherwise.

As noted in the system specification, the details of the rules taken into account for the constitution of the XML file are beyond the scope of this document, but the schema is attached there.

Acknowledgements

Swiss Post is thankful to all security researchers for their contributions and the opportunity to improve the system's security guarantees. In particular, we want to thank the following experts for their reviews or suggestions reported on our [Gitlab repository](#). We list them here in alphabetical order:

- Aleksander Essex (Western University Canada)
- Rolf Haenni, Reto Koenig, Philipp Locher, Eric Dubuis (Bern University of Applied Sciences)
- Thomas Edmund Haines (Australian National University)
- Olivier Pereira (Université catholique Louvain)
- Vanessa Teague (Thinking Cybersecurity)

References

- [1] Die Schweizerische Bundeskanzlei (BK): *Federal Chancellery Ordinance on Electronic Voting (OEV)*, 01 July 2022.
- [2] Die Schweizerische Bundeskanzlei (BK): *Partial revision of the Ordinance on Political Rights and total revision of the Federal Chancellery Ordinance on Electronic Voting (Redesign of Trials). Explanatory report for its entry into force on 1 July 2022.*
- [3] Eidgenössisches Departement für auswärtige Angelegenheiten EDA: *Swiss Political System - Direct Democracy*. <https://www.eda.admin.ch/aboutswitzerland/en/home/politik/uebersicht/direkte-demokratie.html/>. Retrieved on 2020-07-15.
- [4] gfs.bern: *Vorsichtige Offenheit im Bereich digitale Partizipation - Schlussbericht*. Mar. 2020.
- [5] R. Haenni, E. Dubuis, R. E. Koenig, and P. Locher: “CHVote: Sixteen Best Practices and Lessons Learned”. In: *International Joint Conference on Electronic Voting*. Springer. 2020, pp. 95–111.
- [6] R. Haenni, E. Dubuis, R. E. Koenig, and P. Locher: “Process models for universally verifiable elections”. In: *International Joint Conference on Electronic Voting*. Springer. 2018, pp. 84–99.
- [7] S. Josefsson et al.: *The base16, base32, and base64 data encodings*. Tech. rep. RFC 4648, October, 2006.
- [8] B. Smyth: “A foundation for secret, verifiable elections.” In: *IACR Cryptology ePrint Archive* 2018 (2018), p. 225.
- [9] Swiss Post: *Cryptographic Primitives of the Swiss Post Voting System. Pseudocode Specification. Version 1.6.0*. <https://gitlab.com/swisspost-evoting/crypto-primitives/crypto-primitives>. 2026.
- [10] Swiss Post: *Protocol of the Swiss Post Voting System. Computational Proof of Complete Verifiability and Privacy. Version 1.5.0*. <https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/Protocol>. 2026.
- [11] Swiss Post: *Swiss Post Voting System. System Specification. Version 1.6.1*. <https://gitlab.com/swisspost-evoting/e-voting/e-voting-documentation/-/tree/master/System>. 2026.
- [12] *W3C XML Schema Definition Language*. <https://www.w3.org/TR/xmlschema11-2/#token>. 2012.

List of Algorithms

4.1	VerifyOnlineControlComponentsBallotBox	39
4.2	VerifyTallyControlComponentBallotBox	41
4.3	VerifyProcessPlaintexts	42
4.4	VerifyECH0222	43

List of Verifications

0.01	ManualChecksByAuditors	14
1.01	VerifySetupCompleteness	16
2.01	VerifySignatureCantonConfig	17
2.02	VerifySignatureSetupComponentPublicKeys	18
2.03	VerifySignatureControlComponentPublicKeys	18
2.04	VerifySignatureSetupComponentTallyData	19
2.05	VerifySignatureElectionEventContext	19
3.01	VerifyEncryptionGroupConsistency	20
3.02	VerifyNodeIdsConsistency	20
3.03	VerifyFileNameNodeIdsConsistency	20
3.04	VerifyElectionEventIdConsistency	21
3.05	VerifyVerificationCardSetIdsConsistency	21
3.06	VerifyFileNameVerificationCardSetIdsConsistency	21
3.07	VerifyVerificationCardIdsConsistency	22
3.08	VerifyCCRChoiceReturnCodesPublicKeyConsistency	22
3.09	VerifyCCMElectionPublicKeyConsistency	22
3.10	VerifyCCMAndCCRSchnorrProofsConsistency	23
3.11	VerifyChoiceReturnCodesPublicKeyConsistency	23
3.12	VerifyElectionPublicKeyConsistency	23
3.13	VerifyPrimesMappingTableConsistency	24
3.14	VerifyTotalVotersConsistency	25
5.01	VerifyEncryptionParameters	26
5.02	VerifySmallPrimeGroupMembers	27
5.03	VerifyVotingOptions	28
5.04	VerifySchnorrProofs	29
6.01	VerifyTallyCompleteness	30
7.01	VerifySignatureControlComponentBallotBox	31
7.02	VerifySignatureControlComponentShuffle	32
7.03	VerifySignatureTallyComponentShuffle	32
7.04	VerifySignatureTallyComponentVotes	33
7.05	VerifySignatureTallyComponentECH0222	33
8.01	VerifyEncryptionGroupConsistency	34
8.02	VerifyNodeIdsConsistency	34
8.03	VerifyFileNameNodeIdsConsistency	34
8.04	VerifyElectionEventIdConsistency	35
8.05	VerifyBallotBoxIdsConsistency	35
8.06	VerifyFileNameBallotBoxIdsConsistency	35
8.07	VerifyVerificationCardIdsConsistency	36

8.08 VerifyConfirmedEncryptedVotesConsistency	36
8.09 VerifyCiphertextsConsistency	36
8.10 VerifyPlaintextsConsistency	37
8.11 VerifyNumberConfirmedEncryptedVotesConsistency	37
10.01VerifyOnlineControlComponents	38
10.02VerifyTallyControlComponent	40

List of Figures

1	Control component authenticity check	12
---	--	----

List of Tables

1	Verification categories and their description.	10
2	Overview of the auditor's knowledge of the election event parameters	13
3	Context Dataset	16
4	Authenticity checks for setup	17
5	Overview of the generation of voting options	24
6	Tally Dataset	30
7	Authenticity checks for final verification	31